

Simple English Visual Speech Recognition on Embedded Microcontroller

Jason Herrmann

Simple English Visual Speech
Recognition on Embedded
Microcontroller

Phoneme

- Smallest unit of sound in a language
 - /b/ in bat or /p/ in pat

Viseme

- The visual equivalent of a phoneme
 - /p/, /b/, /m/ all involve closing the lips

Visual Speech Recognition (VSR)

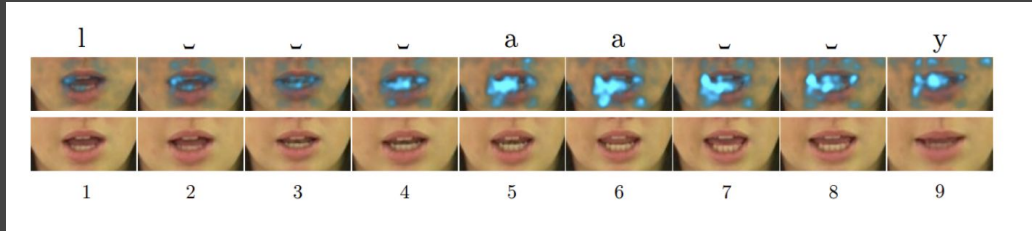
- Lip Reading

STM32H7

Most modern, accurate VSR systems require large deep neural networks and powerful GPU compute

e. g.

- LipNet 2016
 - One of the earliest VSR models
 - Roughly 18MB or 4.6 Quantized
- ResNet 2022
 - ResNet-18 Front end
 - MS-TCN back-end (Multi-Scale Convolutional Network)
 - Roughly 162MB



<https://sds-aau.github.io/M3Port19/portfolio/lipnet/>

<https://arxiv.org/pdf/1611.01599>

Goal

Build a lightweight Visual Speech Recognition (VSR) system

Identify visemes on the STM32H7 at some accuracy

Applications

- Real-time speech understanding without cloud compute
- Audio transcription in noisy environments
 - Surveillance
 - Medical devices (hearing assist)
 - Espionage
 - Video transcription
- I took ECE 114, Speech and Image Processing last quarter



youtube.com



<https://www.pexels.com/security/>

How?

1. Create a custom dataset with limited parameters
2. Use small CNN and quantize

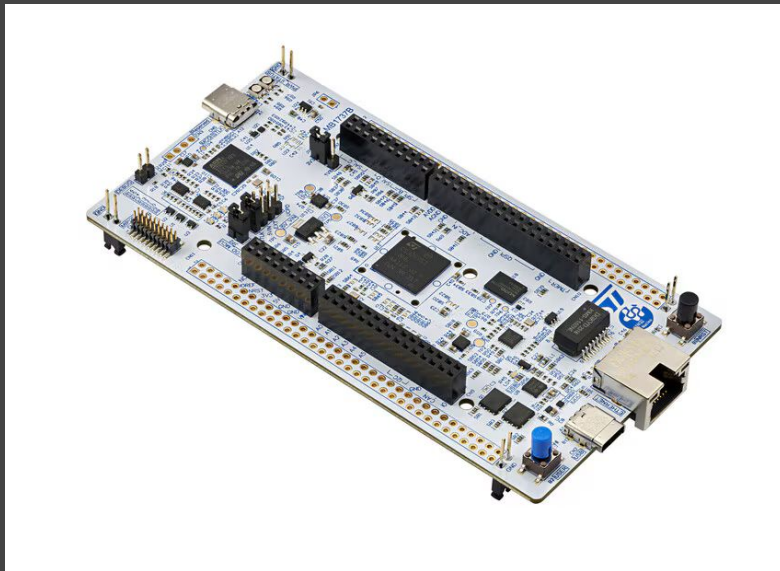
The Challenge

Modern VSR systems:

- Large CNN/RNN models
- Require GPU
- Large Datasets

My constraints:

- Limited Processing
 - Total SRAM: 1 MB
 - Fast Ram: ~128 KB
 - Flash size: 2 MB
- No GPU processing



<https://www.st.com/en/microcontrollers-microprocessors/stm32h7-series.html>

1 Create a custom dataset with limited parameters

Problem: Large VSR systems do whole word classification or complex sequence modeling

Solution: Split every sound into its phoneme, classify the visually similar ones

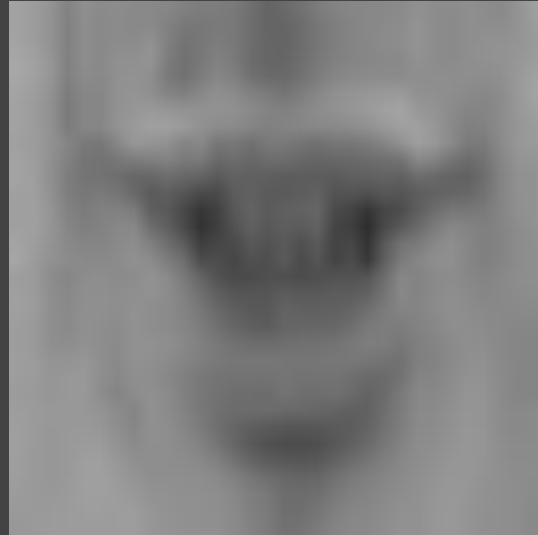
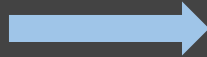
ɑ	ɐ	ɒ	æ	ɪ	ʊ	β	ç	ε	ð
ɔ	ɔ̃	ɛ	ɜ	ə	ɜ̃	ɝ	ɞ	ɣ	ɹ
ɻ	ɸ	ɦ	ħ	ʰ	ŋ	ɪ	ɨ	ɩ	ɰ
ʝ	ʃ	ʒ	ʝ	ɬ	ɭ	ɮ	ɯ	ɰ	ɱ
ɱ	ɱ	ɱ	ɱ	ɱ	ɱ	ɱ	ɱ	ɱ	ɱ
ø	œ	œ	ϕ	ɹ	ɹ	ɹ	ɹ	ɹ	ɹ
ɹ	ɹ	ɹ	θ	ɬ	ɭ	ɮ	ɯ	ɰ	ɱ
ʷ	χ	ʎ	ɣ	ɰ	ɮ	ɮ	ɮ	ɮ	ɮ
ʔ	ʔ	ʔ	ʔ	ʔ	ʔ	ʔ	ʔ	ʔ	ʔ
ɖ	ɖ	ɖ	ɖ	ɖ	ɖ	ɖ	ɖ	ɖ	ɖ
ɖ	ɖ	ɖ	ɖ	ɖ	ɖ	ɖ	ɖ	ɖ	ɖ
ɹ	ɹ	ɹ	ɹ	ɹ	ɹ	ɹ	ɹ	ɹ	ɹ

<https://www.fon.hum.uva.nl/praat/>

1 Create a custom dataset with limited parameters

Problem: Large VSR systems use whole 32-bit RGB videos

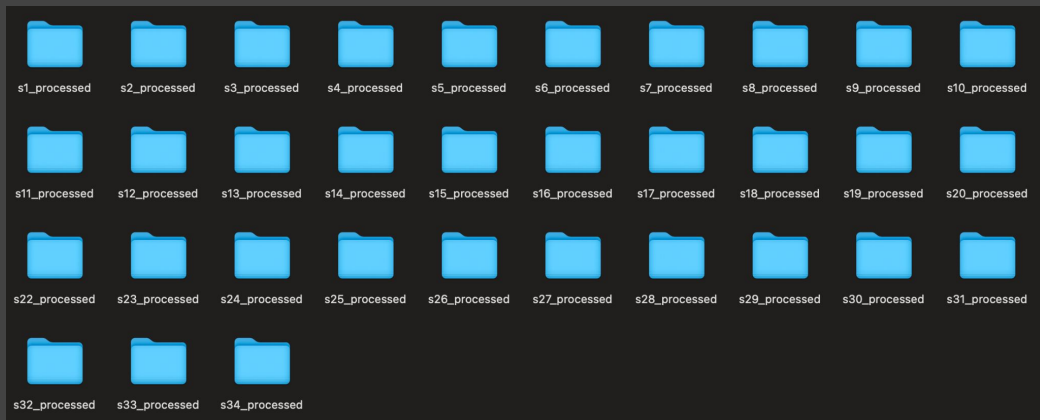
Solution: Crop just the mouth, use grayscale values, reduce resolution, reduce frame sizes



1.a Obtain a dataset

I chose GRID Corpus

- Free, open-source
- Consistent



<https://spandh.dcs.shef.ac.uk/gridcorpus/>



align



bbaf2n.mpg



bbaf3s.mpg



bbaf4p.mpg



bbaf5a.mpg



bbal6n.mpg



bbal7s.mpg



bbal8p.mpg



bbal9a.mpg



bbas1s.mpg



bbas2p.mpg



bbas3a.mpg



bbaszn.mpg



bbaz4n.mpg



bbaz5s.mpg



bbaz6p.mpg



bbaz7a.mpg



bbbf6n.mpg



bbbf7s.mpg



bbbf8p.mpg



bbbf9a.mpg



bbbm1s.mpg



bbbm2p.mpg



bbbm3a.mpg



bbbmzn.mpg



bbbs4n.mpg



bbbs5s.mpg



bbbs6p.mpg



bbbs7a.mpg



bbbz8n.mpg



bbbz9s.mpg



bbbie8n.mpg



bbbie9s.mpg



bbbif1a.mpg



bbbifzp.mpg



bbbil2n.mpg



bbbil3s.mpg



bbbil4p.mpg



bbbil5a.mpg



bbbir6n.mpg



bbbir7s.mpg



bbbir8p.mpg



bbbir9a.mpg



bbbiz1s.mpg



bbbiz2p.mpg



bbbiz3a.mpg



bbbizzn.mpg



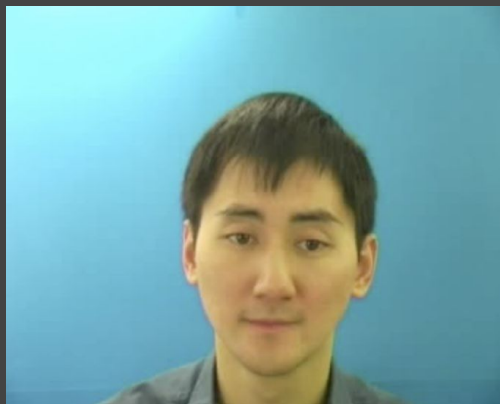
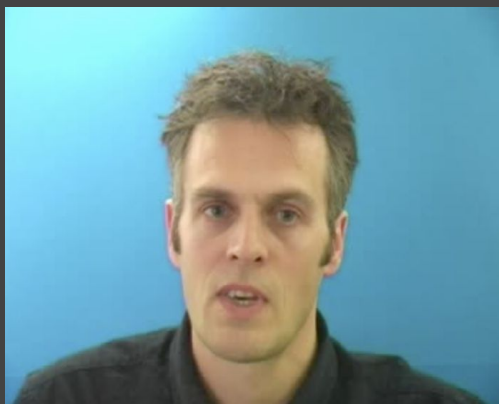
bbbwg1s.mpg



bbbwg2p.mpg

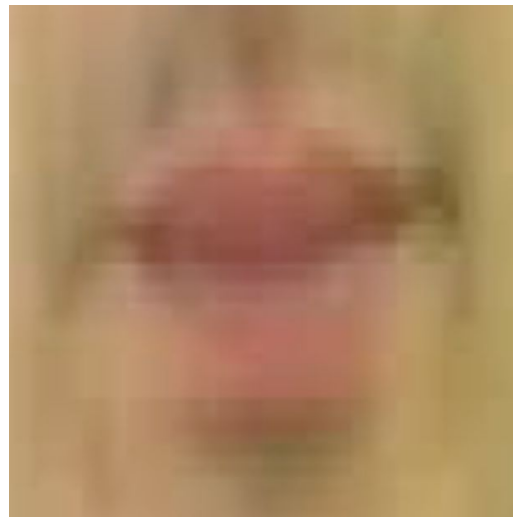
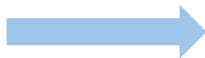
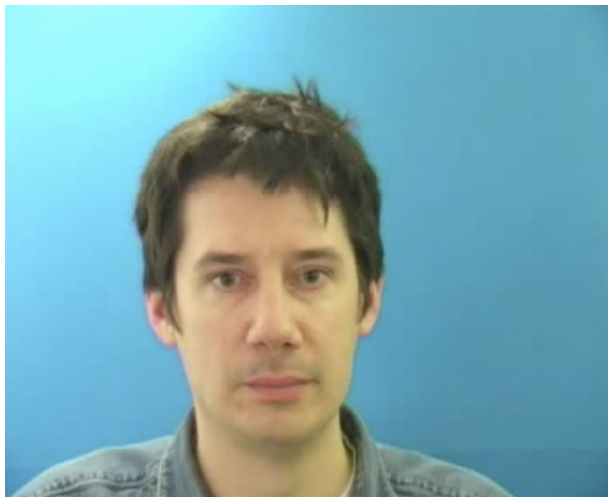


bbbwg3a.mpg



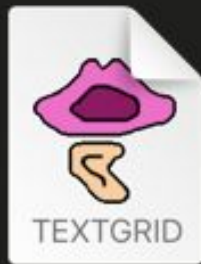
- MediaPipe

- Developed by google
- OpenCV to load each frame
- Detects facial landmarks

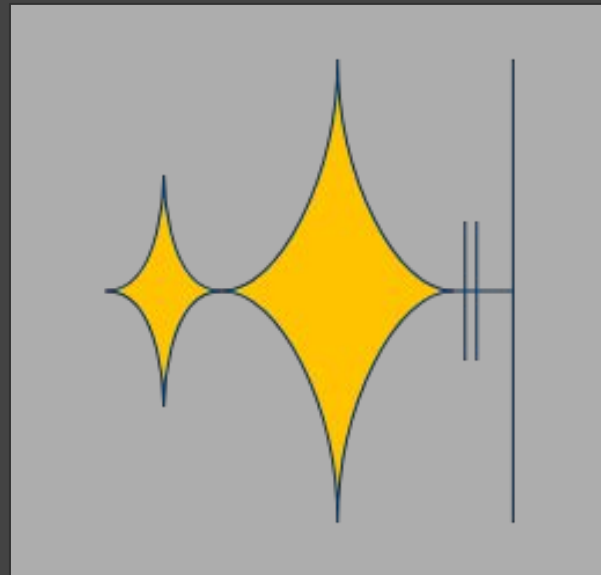


1.c Phoneme Timestamps

- Montreal Forced Aligner (MFA)
 - CORPUS tool
 - Ran in Conda
 - Requires:
 - Text transcript, audio file



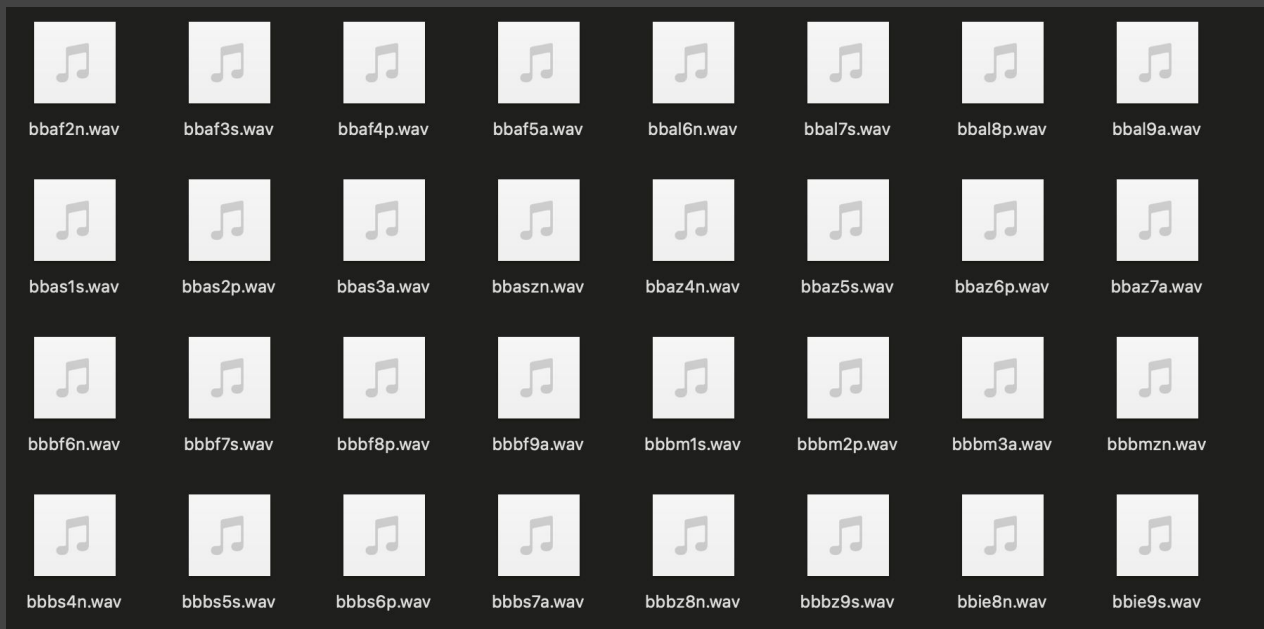
bbaf2n.TextGrid



<https://montreal-forced-aligner.readthedocs.io/en/latest/>

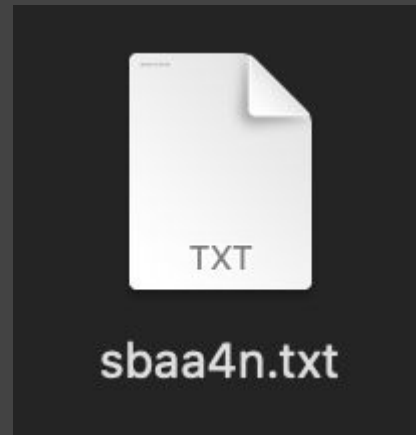
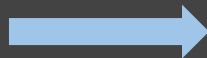
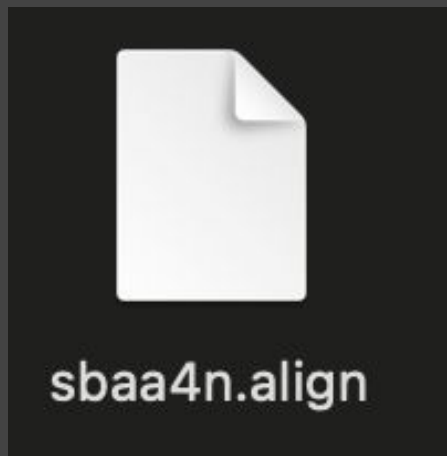
1.c.a Phoneme Timestamps

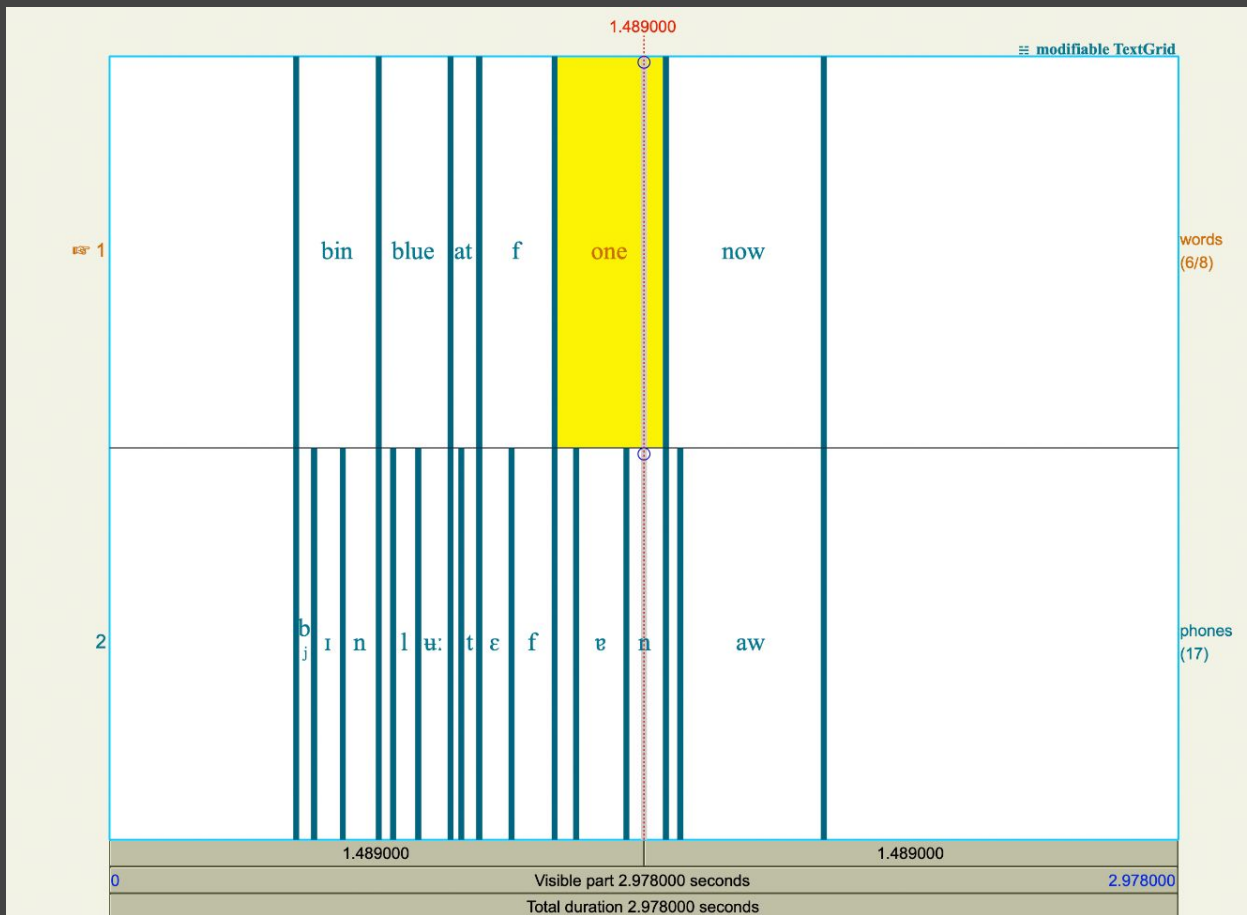
- Take WAV files from each video



1.c.b Phoneme Timestamps

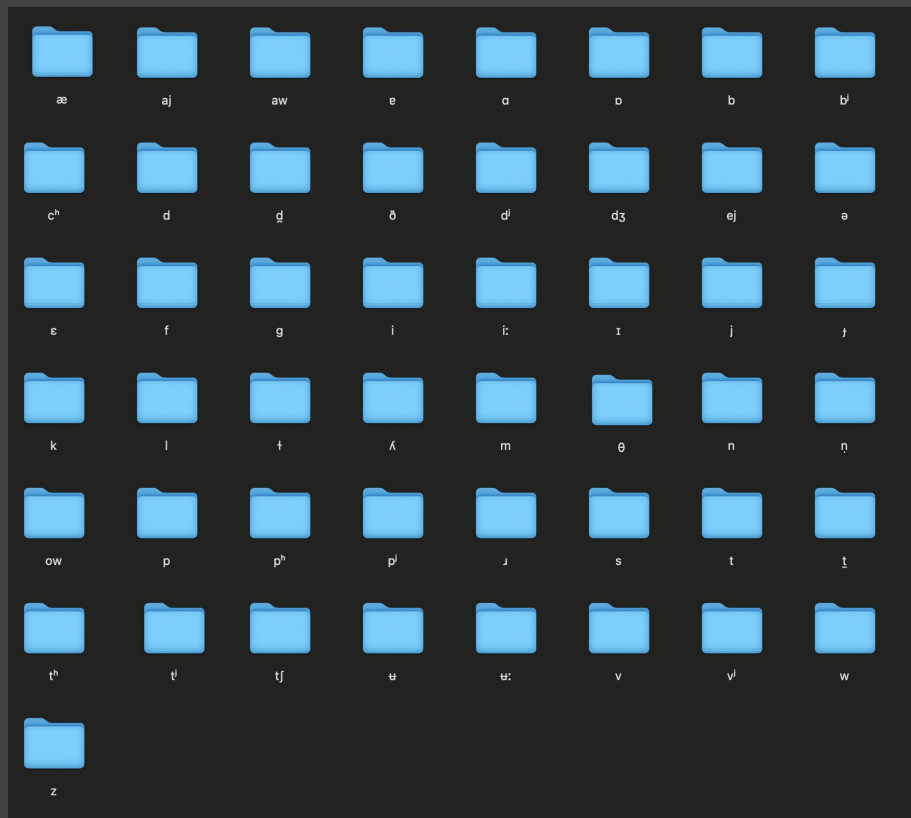
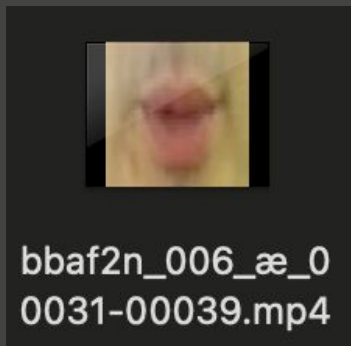
- Take .txt transcripts from GRID dataset





1.d Split the cropped video

- Using the MFA textgrids
- Sort them by their phoneme
- Limit to 9, 7, 5 frames
- Adjust sizes
 - 28x28, 32x32, 48x48



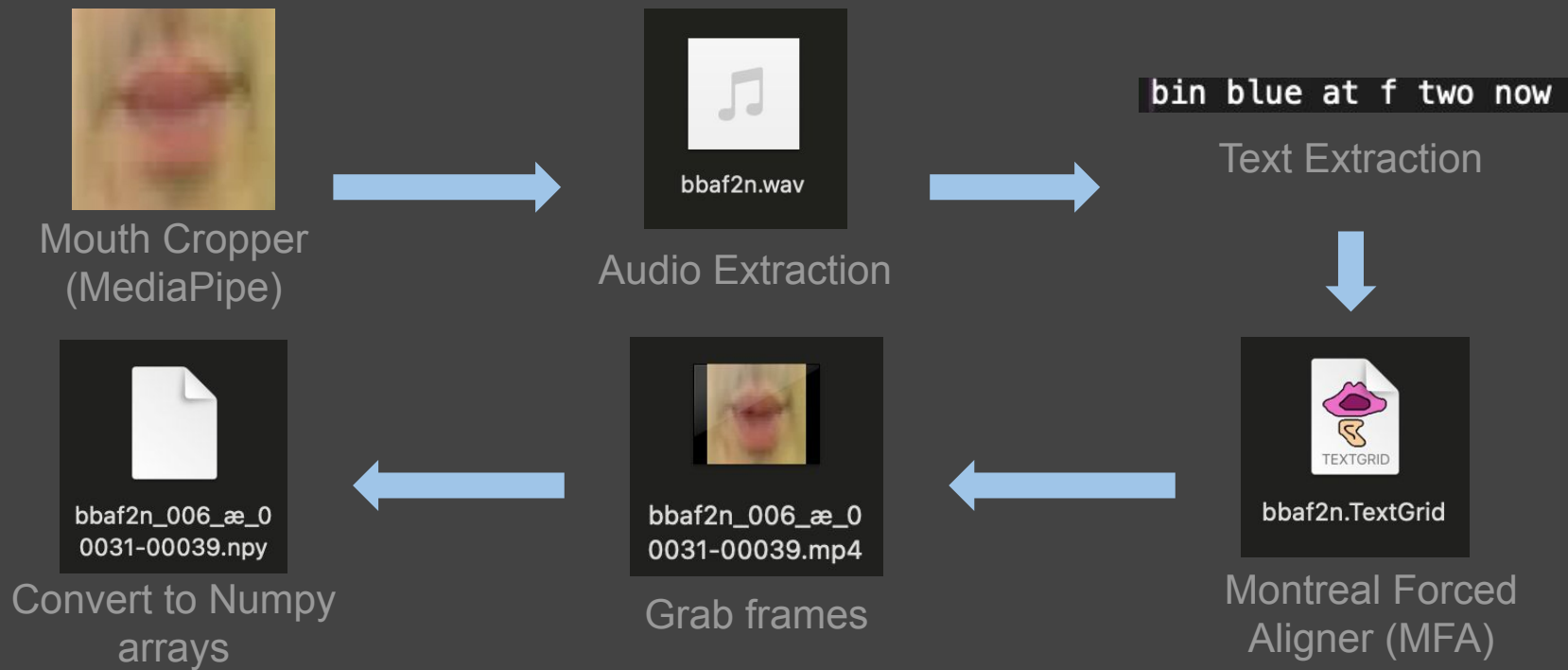
1.e Grayscale, convert to .npy array

- Intensity only (less channels)
- Instead of a 32x32 video of 9 frames it's now a 32x32x9 array
 - The entire sequence is now one file



bbaf2n_006_æ_0
0031-00039.npy

Preprocessing Overview



2 Use a small CNN and quantize

Problem: Large VSR systems use 3D Convolution layers with massive parameters, lots of epochs

Solution: Stick with 2D and assess accuracy at different parameters to maximize accuracy with less parameters

2 Use a small CNN and quantize

For Clarity: Assess at many different sizes, frame depths, and weights

2.a Choose 5 Viseme Classes

- Use distinct visemes

```
#viseme groups
```

```
BILABIALS = {"b", "bʲ", "p", "pʰ", "pʲ", "m"}
```

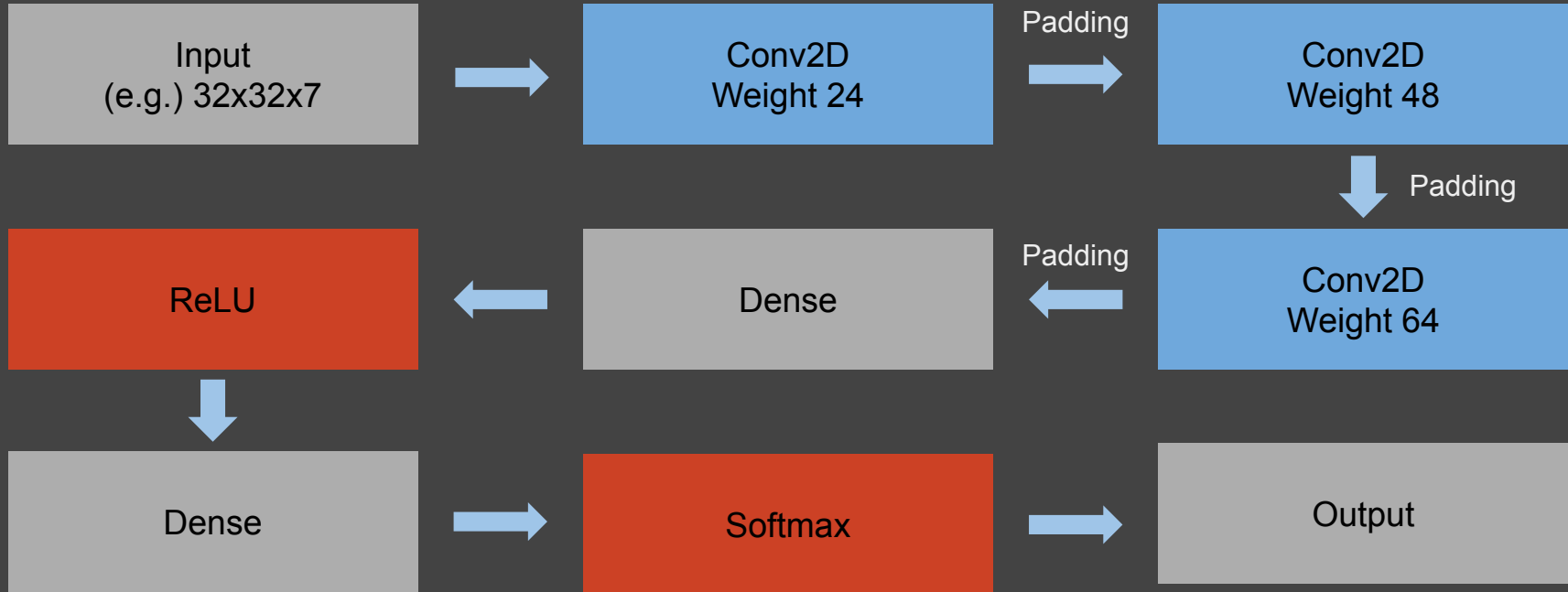
```
FRICATIVES = {"s", "z", "tʃ", "dʒ"}
```

```
ROUND = {"ow", "ʊ", "ʊː"}
```

```
SPREAD = {"i", "i̟", "ɪ̟"}
```

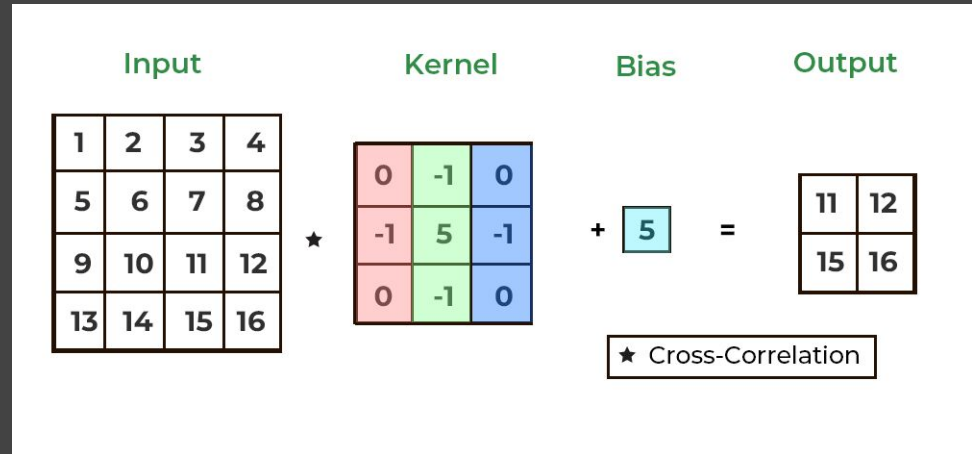
```
LABIODENTALS = {"f", "v", "vʲ"}
```


2.b CNN Model Architecture



2.b.a Conv2D

- 3x3 kernel across # of layers
- Stride 1
- Padding = “same”
- Identifies features



<https://www.geeksforgeeks.org/computer-vision/apply-a-2d-convolution-operation-in-pytorch/>

2.b.a Conv + BatchNorm + ReLU

After the convolution, BatchNorm centers and normalizes data around zero and ReLU clips negatives

$$\mu_{batch} = \frac{1}{m} \sum_{i=1}^m x_i$$

ReLU

$$f(x) = \max(0, x)$$

2.b.b Density

- Weighted sum of all outputs
- Combine features

$$y_i = \sum_{j=1}^n w_{ij} x_j + b_i$$

2.b.c Softmax

Probabilities for M, SH, O, E, FV

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{i=1}^5 e^{z_i}}$$

2.b.d Training Split

- Training set
 - Learns weights
- Validation set
 - Used to detect overfitting
- Test set
 - Validation and accuracy

80-20

2.b.e Model sizes

Number of filters per 2D convolution layer

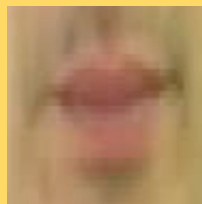
More filters, the more detailed the feature maps

```
filters_map = {  
    "small": [16, 32, 48],  
    "medium": [24, 48, 64],  
    "large": [32, 64, 96],  
}
```

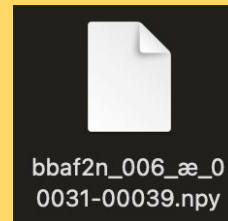
System Overview



Video



Lip detection +
cropping



Extract Phonemes



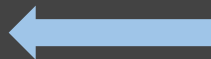
```
#viseme groups
BILABIALS = {"b", "bʰ", "p", "pʰ", "pʲ", "m"}
FRICATIVES = {"s", "z", "tʃ", "dʒ"}
ROUND = {"ow", "ʊ", "ʉ"}
SPREAD = {"i", "i̥", "i̜"}
LABIODENTALS = {"f", "v", "vʲ"}
```

CNN Classifier



test_loss=0.730552
test_accuracy=0.807874

Prediction



Embedded
Result

Testing Sh...

ai_run_one start

before buffer assign

ai_input=0x24006c98

ai_output=0x24006ccc

before ai_network_run

after ai_network_run, batch=1

Output scores:

M: -128

SH: 123

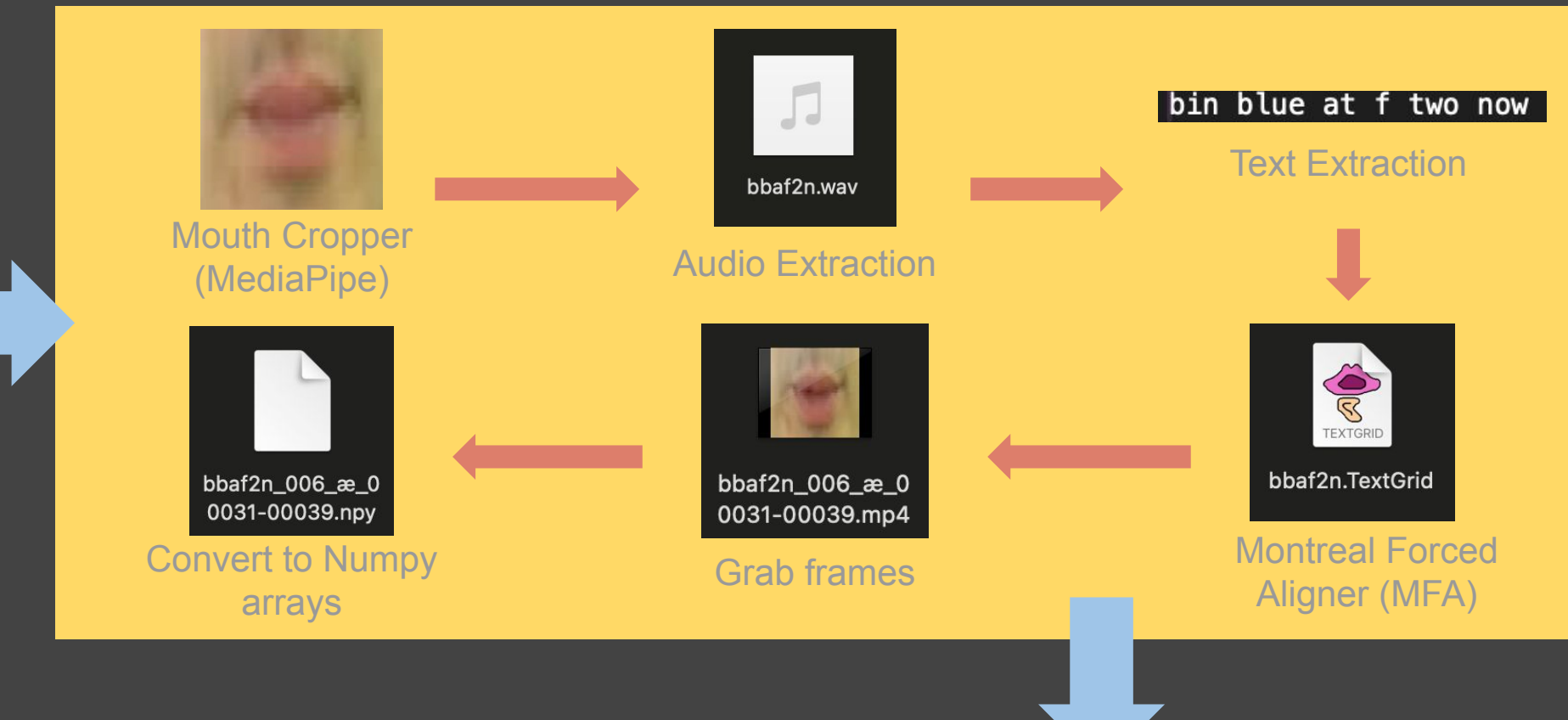
O: -126

E: -126

FV: -126

Predicted: SH

Preprocessing Overview

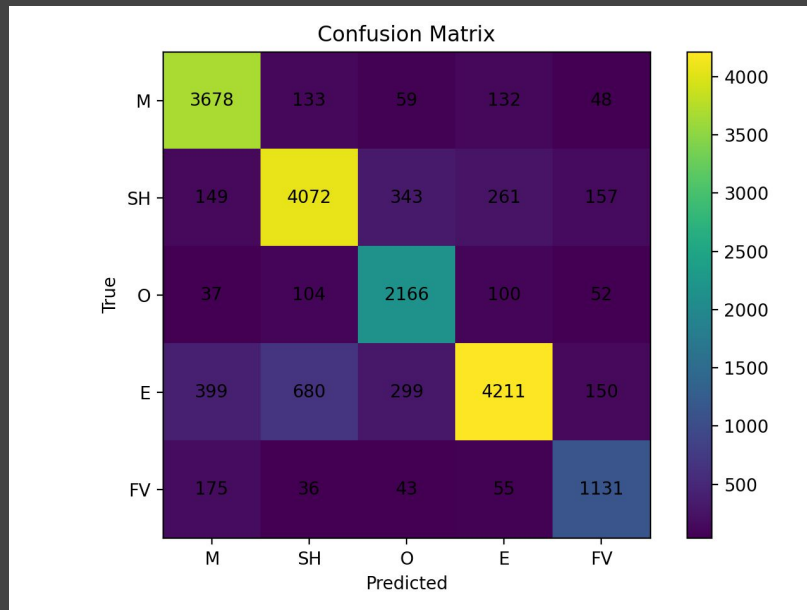


CNN Results

Image Size	Avg Accuracy
28x28	77.7%
32x32	76.5%
48x48	72.1%

	precision	recall	f1-score	support
M	0.8288	0.9081	0.8666	4050
SH	0.8103	0.8173	0.8138	4982
O	0.7443	0.8808	0.8069	2459
E	0.8848	0.7338	0.8022	5739
FV	0.7354	0.7854	0.7596	1440
accuracy			0.8172	18670
macro avg	0.8007	0.8251	0.8098	18670
weighted avg	0.8228	0.8172	0.8166	18670

Best result: 28x28x7, Medium 81.72% accuracy



Quantization

I exported my model as an INT8 TFLite model from the Python keras model

Then the X-Cube-AI generated quantized weights and rescaling logic

All I had to do was quantize the input arrays

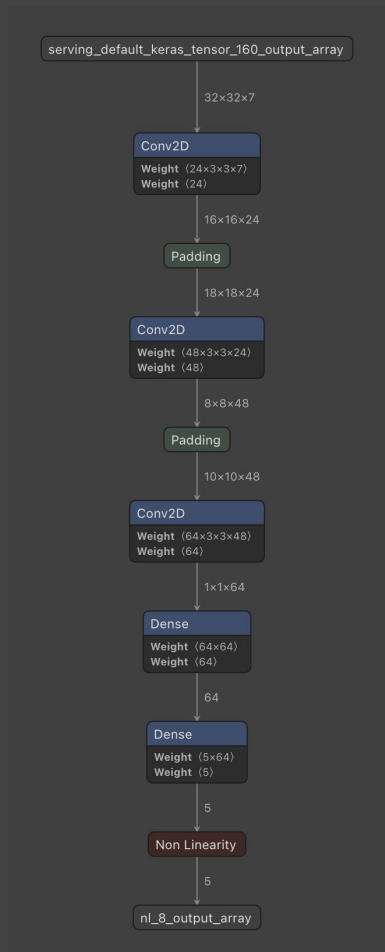
Embedded Deployment

- Tensorflow → TFLite
- Quantization (INT8)
- X-Cube-AI

Model Size

Flash: 76 KB / 2048 KB = 3.7%

RAM: 20 KB / 1024 KB = 2%



Embedded Prediction

Dataset Example:

Sh

Scores:

M: -128

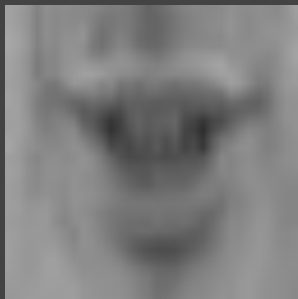
SH: 123

O: -126

E: -126

FV: -126

Predicted: SH



Personal Example:

O

Scores:

M: -128

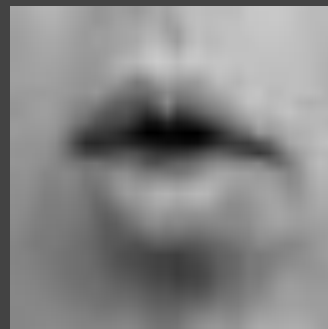
SH: -126

O: 75

E: -77

FV: -128

Predicted: O



Shortcomings

- This model only works for: specific lighting, at specific angles, on specific people
- Dataset bias is heavily shifted towards fair skin
- The dataset is also all british speaking

For the future

- Add more diversity
- Add more variance in lighting and speaker angle
- Increase viseme class count
- 3D CNN
- Live Demo

Conclusion

Simple English VSR on Constrained embedded hardware is possible but difficult.

It does not require GPU systems, but achieving it on small embedded hardware requires careful engineering tradeoffs